

Senior .NET Interview Pitch Sheet

Concentrado de preguntas clave y respuestas Senior en inglés — Dental Clinic PlayBook

MODULE A — ARCHITECTURE & EF CORE FOUNDATIONS

Composition vs Inheritance

Q: Why did you model Patient and Doctor using composition instead of a shared Person base class?

A: Patient and Doctor aren't variants of the same concept — they're independent domain aggregates with different invariants. Modeling them with a shared abstract Person class or an IPerson interface creates real problems in EF Core: Table-Per-Hierarchy produces a single table riddled with NULL columns, while Table-Per-Type forces expensive JOINS on every read. Instead, I apply a Has-A relationship: both Patient and Doctor contain a PersonName and ContactInfo value object. This gives zero JOINS, zero NULL contamination, and centralizes validation logic in the value objects themselves, all while keeping each aggregate's schema flat and fast to query.

N+1 Query Problem

Q: What is the N+1 problem and how do you prevent it in EF Core?

A: The N+1 problem occurs when an initial query fetches N parent records, and subsequent code accesses navigation properties, triggering N additional database queries in a loop. I prevent it using DTO Projections via .Select() and Eager Loading via .Include(), while disabling Lazy Loading across the DbContext so any accidental N+1 access fails fast in integration tests rather than silently degrading production performance.

IQueryable vs IEnumerable

Q: What's the difference between IQueryable and IEnumerable, and why does it matter for performance?

A: IQueryable builds an expression tree that gets translated into a single optimized SQL query at materialization time — filtering, sorting, and pagination all execute inside SQL Server. IEnumerable operates on objects already loaded into application memory using compiled C# delegates. Converting an IQueryable to IEnumerable before filtering forces EF Core to pull the entire table into RAM first, which is a common cause of OutOfMemoryExceptions on large tables. On the read side of CQRS, I strictly enforce IQueryable composition with .AsNoTracking().Select(...), ensuring SQL engine pushdown with minimal RAM overhead.

MODULE B — INTERFACES & ABSTRACTION

1. Interfaces

Q: What is an interface and why use one instead of a concrete class dependency?

A: An interface defines a behavioral contract without dictating implementation, which lets consuming code depend on an abstraction rather than a concrete class. In the dental clinic system, `TreatmentPlan` depends on `INotificationSender` instead of a specific `EmailNotificationSender`, so I can introduce new notification channels — SMS, WhatsApp — without modifying or recompiling the service that consumes them. This directly supports the Open/Closed Principle and makes the class trivially testable, since I can substitute a mock implementation in unit tests. Technically, this is only possible because mocking frameworks generate dynamic proxies — they can implement any interface freely, but can't override non-virtual methods on a concrete class.

2. Abstract Classes

Q: When would you use an abstract class instead of an interface?

A: An abstract class lets me share real, concrete implementation across related classes while still forcing each subclass to provide its own specific behavior through abstract methods. In the dental clinic notification system, `NotificationSenderBase` implements retry logic exactly once using the Template Method pattern, while `EmailNotificationSender` and `SmsNotificationSender` only override the piece that's actually different — how the message physically gets sent. An interface defines what must be done with no shared code; an abstract class defines what must be done differently alongside what should behave identically across implementations.

3. Interface vs Abstract Class

Q: How do you decide between an interface and an abstract class in a real design?

A: I choose based on relationship, not just syntax. If two classes are fundamentally the same kind of thing and share real implementation — like `EmailNotificationSender` and `SmsNotificationSender`, which both need identical retry logic — an abstract class avoids duplicating that behavior. But if classes are unrelated in identity and only need to share a capability — like `Patient` and `Dentist`, independent aggregates that both need to export a PDF profile — an interface is the right tool, since it expresses 'can do X' without forcing an artificial shared ancestor. I also keep consuming code depending on interfaces rather than abstract classes whenever possible, keeping the contract minimal.

4. Virtual / Override / Overload

Q: What happens if you try to override a non-virtual method, and how does that differ from method hiding?

A: Without `virtual` on the base method, `override` in the derived class produces a compile-time error — CS0506 — because `override` requires an explicit contract from the base class saying the method can be replaced. If you use `new` instead, the code compiles, but you lose runtime polymorphism: the compiler resolves which method to call based on the declared type of the variable, not the actual object type, silently producing wrong behavior when working through a base class reference. This distinction — method overriding vs. method hiding — is a common source of subtle bugs in inheritance hierarchies. `Overload`, by contrast, has nothing to do with inheritance at all — it's resolved entirely at compile time based on the method signature.

5. Readonly / Constant

Q: Why can't you declare a const DateTime, and what would you use instead?

A: const requires a value the compiler can embed directly into the compiled IL at compile time — only literal values of primitive types, with zero runtime execution. DateTime.UtcNow is a method call that reads the system clock, so its value doesn't exist until the program runs. Even a fixed DateTime isn't valid for const, because constructing a DateTime struct still requires executing code at runtime; DateTime simply isn't a const-eligible type in C#, regardless of whether the value is fixed. For values like this, I use static readonly instead — it's still assigned only once, but the assignment happens at runtime during type initialization, which is exactly what a computed or constructed value needs.

6. Multiple Interface Implementation

Q: Would you design one large capabilities interface or several small ones, and why?

A: I'd use several small interfaces rather than one large fat interface, because the two designs behave very differently once a second class enters the picture. If Patient needs ExportProfileToPdf and IAuditable, but never authentication, a fat-interface design forces Patient to implement Authenticate anyway — either throwing NotSupportedException at runtime or faking a meaningless implementation. That's a runtime problem masquerading as a compile-time contract. With small, focused interfaces, Patient simply implements what it needs and never touches what it doesn't — the compiler enforces exactly what each class can do, with zero dead or dishonest methods. This is the Interface Segregation Principle in practice: no class should be forced to depend on behavior it doesn't use.

7. Default Interface Members

Q: What problem do default interface members solve, and what's the catch?

A: Default interface members, introduced in C# 8, let an interface ship with a default implementation for a method. The main use case is evolving a public contract without breaking existing implementers — if I add a new method to an interface already implemented by ten classes in production, adding it without a default breaks all ten. With a default implementation, they keep compiling and inherit the default automatically, and only the classes that need custom behavior override it. One caveat: the default member is only accessible through the interface type, not directly on the concrete class — but since we already program against interfaces rather than concrete types throughout the codebase, that limitation never actually surfaces in practice.

8. Programming Against Abstractions

Q: Why return IReadOnlyDictionary instead of Dictionary from a service method?

A: I'd return IReadOnlyDictionary instead of Dictionary — the concrete Dictionary type exposes mutation methods like .Add() and .Remove(), so any caller could modify the service's internal collection without it knowing. IReadOnlyDictionary exposes exactly what the caller needs — read access — and nothing more. Separately, I'd also question whether returning the domain entity itself is appropriate at all; for a read-heavy query, I'd project to a lightweight DTO instead. Those are two independent decisions: the collection type controls what operations are allowed, while the DTO decision controls what data is exposed.

9. Dependency Inversion

Q: What is Dependency Inversion, and how does it differ from Dependency Injection?

A: Dependency Inversion has two parts that are often forgotten together: high-level modules shouldn't depend on low-level modules — both should depend on abstractions — and abstractions themselves shouldn't leak implementation details. In our architecture, `CommandHandler` never references `EF Core` directly; it depends on `IApplicationDbContext`, and the concrete `ApplicationDbContext` implements that same interface. Neither side depends on the other directly. It's important to distinguish this from Dependency Injection — DIP is the design principle about direction of dependency, while DI is just the mechanical technique, usually via constructor injection and a container, that makes applying DIP practical at scale.

QUICK REFERENCE — DECISION CHEAT SHEET

Situation	Choose
Classes share identity + real behavior	Abstract Class
Classes are unrelated but share a capability	Interface
Need to mock a dependency in unit tests	Depend on Interface
Evolving a public contract without breaking consumers	Default Interface Member
Exposing a collection you don't want mutated externally	ReadOnly* type
High-level module needs a low-level detail	Depend on abstraction (DIP)
Interface has methods some implementers don't need	Split it (ISP)
Value known at compile time, same for everyone	const
Value computed/assigned once at runtime	readonly / static readonly